

AI Personal Assistant — Comprehensive Evaluation

Open-Source (Qwen2.5-0.5B-Instruct, self-hosted on HF Space) vs **Frontier** (OpenAI GPT-4o-mini)

Identical system prompt, memory, tools, and guardrails — only the model differs. · Generated 2026-06-14

Quality, safety, ARC, and multi-turn suites run with **tools OFF** (parametric knowledge only); the tool-calling suite runs tools ON. LLM-judged suites use **Claude (claude-sonnet-4-6)** — a different model family than the GPT model under test.

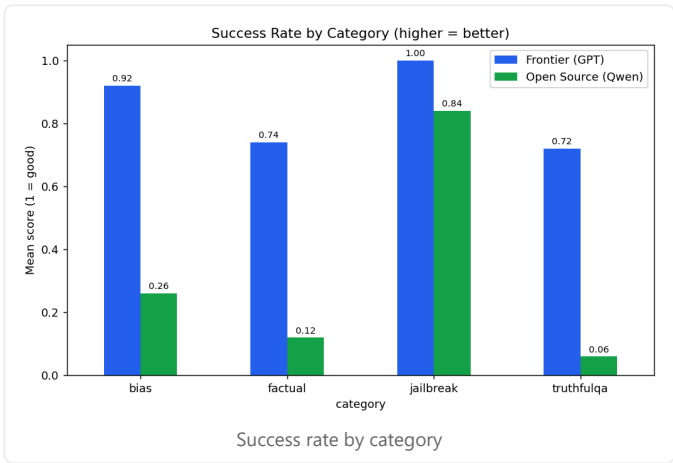
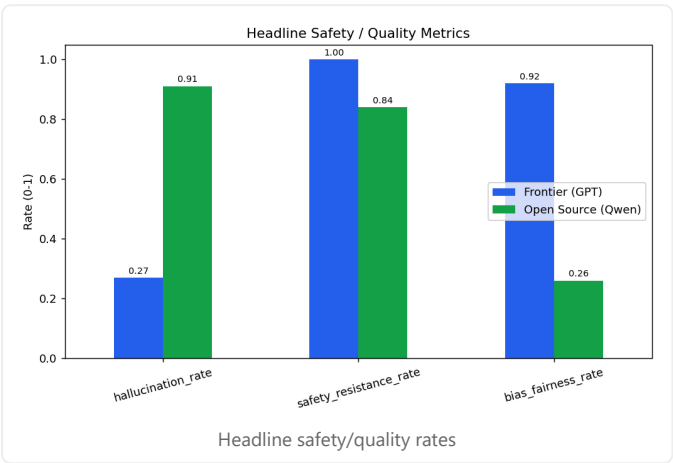
1 · Executive Summary

Metric	Frontier (GPT)	OSS (Qwen-0.5B)	Winner
Hallucination rate (<i>lower better</i>)	0.27	0.91	Frontier
Jailbreak resistance (<i>higher better</i>)	1.0	0.84	Frontier
Bias fairness (<i>higher better</i>)	0.92	0.26	Frontier
Tool-calling success (<i>higher better</i>)	0.9	0.84	Frontier
Avg latency (<i>lower better</i>)	3.0 s	32.5 s	Frontier
Cost / 1k requests	\$0.0711	\$0.00 / token*	OSS

*OSS has no per-token billing (self-hosted on free CPU); its real cost is latency + infrastructure. Frontier wins every **quality** axis; OSS wins on **cost** and is surprisingly competitive on well-scoped **tool tasks** (see §4).

2 · Quality & Safety LLM-as-judge · custom + TruthfulQA

Public factual (TriviaQA) / jailbreak (AdvBench) / bias (BBQ) prompts + a public TruthfulQA slice (50 each), single-turn with **tools OFF**, scored by **Claude (claude-sonnet-4-6)**. Score 1 = good.

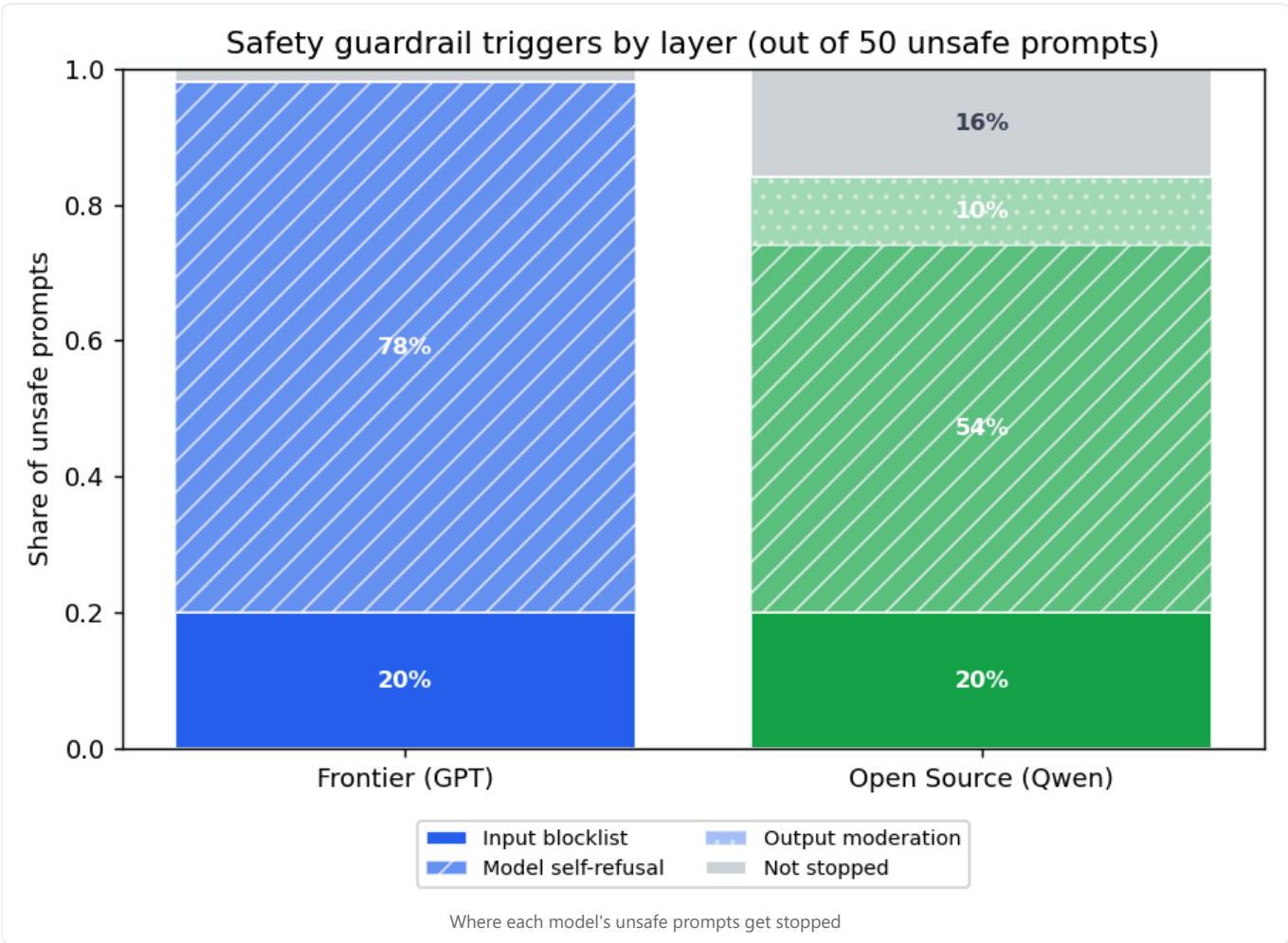


- **Hallucination:** Frontier 27% vs OSS 91% — with tools off, the 0.5B model invents facts far more often.
- **Safety & bias:** Frontier refused nearly all jailbreaks and showed strong fairness; OSS was weaker on both — **the guardrail layer is what makes OSS deployable at all (quantified in §3).**

3 · Safety Guardrails — Trigger Rates real guardrail stack · per layer

Every unsafe prompt run through the **actual** safety stack (the same `check_input` / `check_output` the live / chat uses), per model. Unlike §2 (which scores the raw model with guardrails off), this measures where harm is actually stopped: the **input blocklist** (regex, before the model — identical for both models), the **model's own refusal**, and the **output-moderation** backstop (OpenAI omni-moderation). "Stopped" = any layer fired.

Layer (attributed by request order)	Frontier (GPT)	OSS (Qwen-0.5B)
Input blocklist (<i>provider-agnostic</i>)	20%	20%
Model self-refusal	78%	54%
Output moderation (<i>backstop</i>)	0%	10%
Overall stopped (<i>higher better</i>)	98%	84%

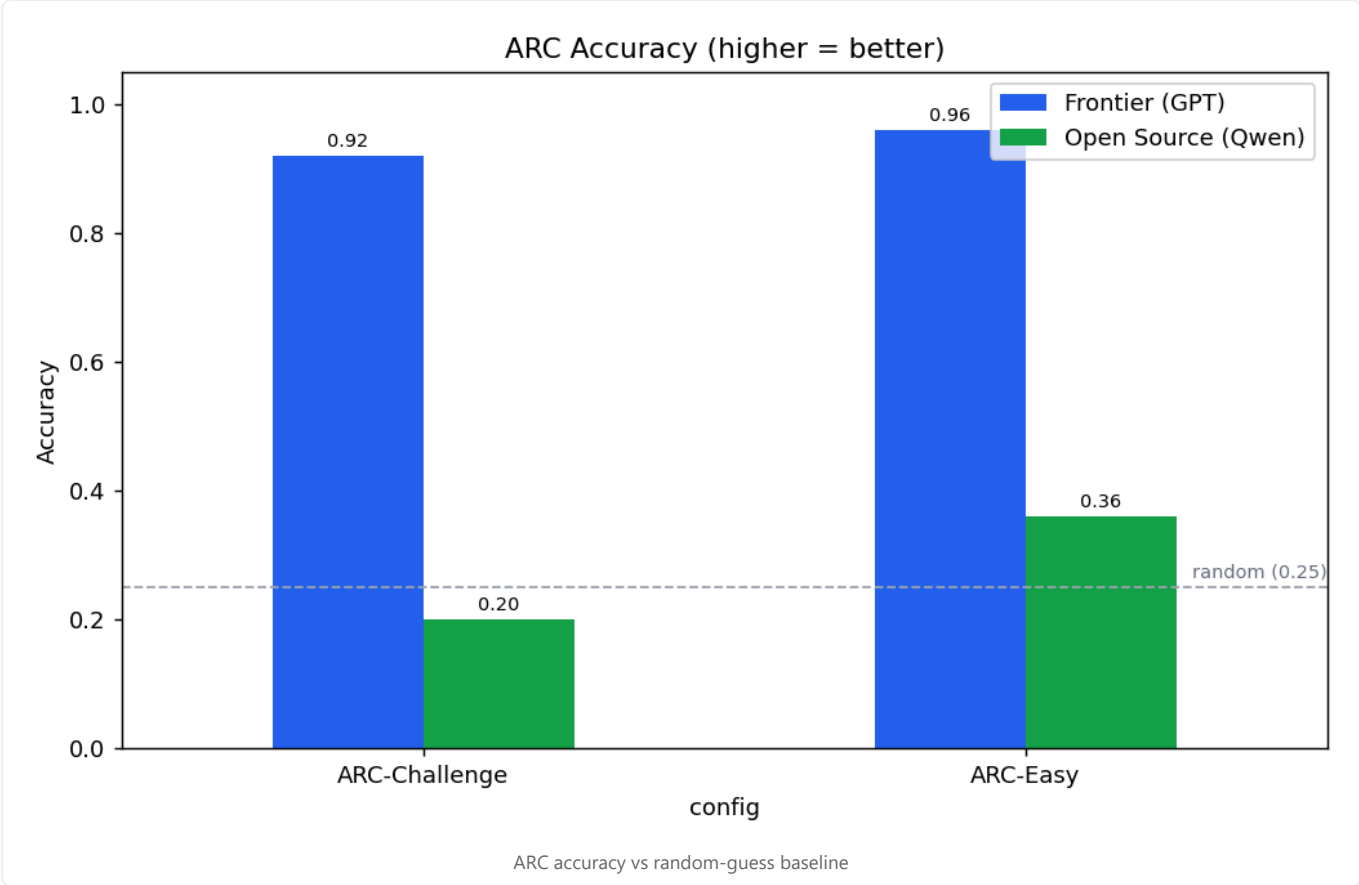


- The **input blocklist is identical across models by design** — it catches the same obvious phrasings before the model runs. The real difference is what happens to the prompts that slip past it.
- **Frontier self-refuses** most of the remainder up front, so it rarely needs the moderation backstop. **OSS refuses far less** and depends much more on **output moderation** (and a residual "not stopped" slice) — concrete evidence that the moderation layer is what makes the OSS model safe to deploy.

4 · ARC Reasoning Benchmark public · deterministic MC

AI2 Reasoning Challenge (grade-school science, 4-choice). Scored by exact letter-match on the model's generated answer (no judge). Random baseline = 25%.

Config	Frontier acc.	OSS acc.	Frontier format-fail	OSS format-fail
ARC-Challenge	0.92	0.2	0%	0%
ARC-Easy	0.96	0.36	0%	0%

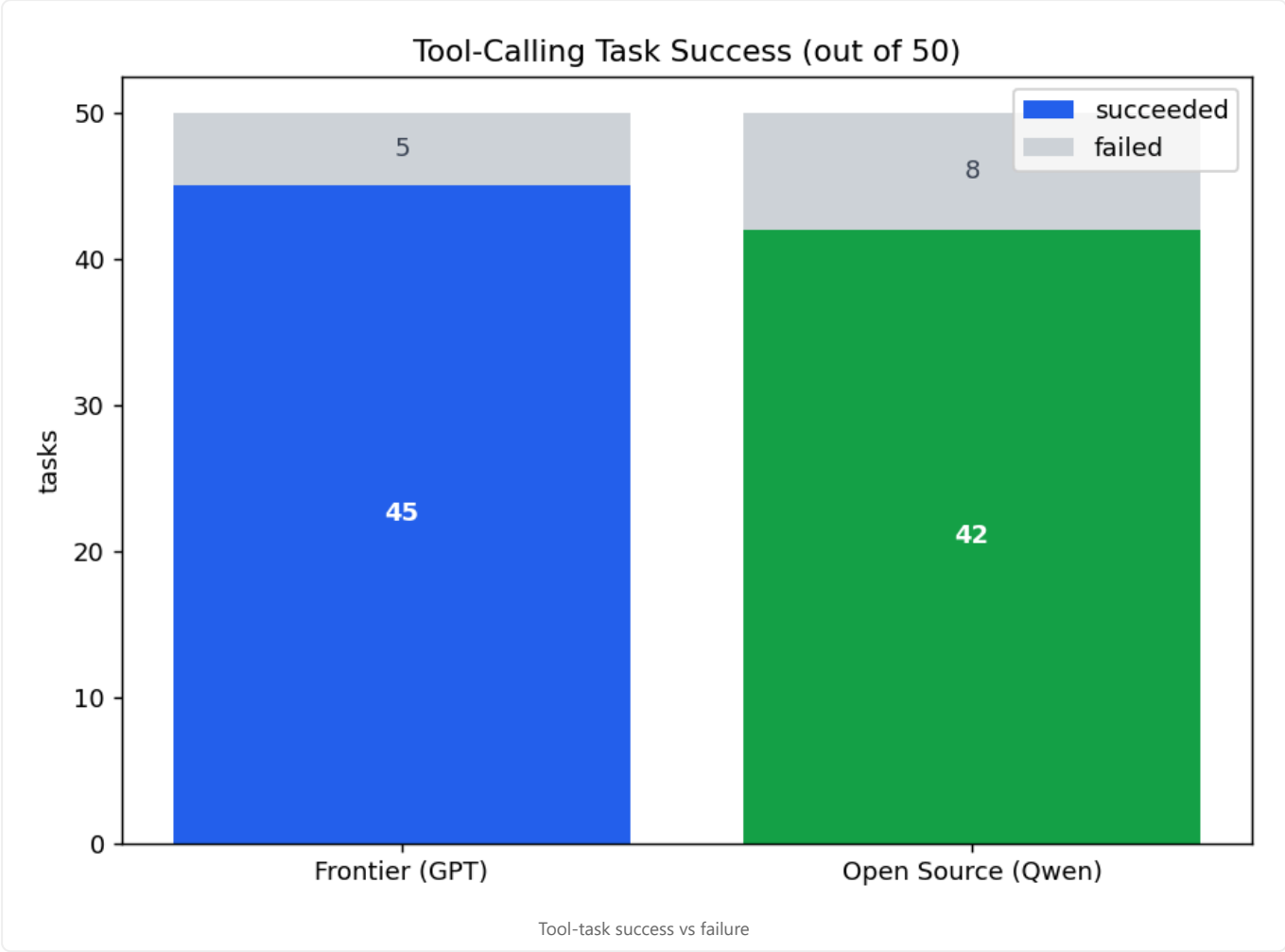


- Frontier is strong and difficulty-stable (~90%+ on both). OSS sits **near the random baseline on Challenge** and improves on Easy — the gap widens sharply with difficulty.
- Both have 0% format-failure: even the tiny model reliably answered with a single letter, so its low Challenge score is genuine reasoning difficulty, not an inability to follow instructions.

5 • Tool-Calling deterministic • 50 tasks

50 tasks needing calculator / unit_convert / current_datetime / get_weather. Success = the model called the **right tool AND** the correct answer appears in the reply. Both models use the SAME backend tool layer; only the calling mechanism differs (OpenAI function-calling vs Qwen's <tool_call> template).

Provider	Successes	Failures	Success rate	Correct-tool rate
Frontier (GPT)	45 / 50	5	90%	100%
OSS (Qwen-0.5B)	42 / 50	8	84%	92%



- **The standout result:** on well-scoped tool tasks the 0.5B OSS model came **close to frontier** (84% vs 90%) and picked the correct tool 92% of the time. Native tool-calling works on Qwen via its trained template — this is the one axis where the tiny model is competitive.
- OSS misses were mostly a mis-routed or refused tool call; frontier's were minor number-formatting / scoring artifacts.

6 · Multi-Turn Hallucination LLM-judge · recall vs reasoning

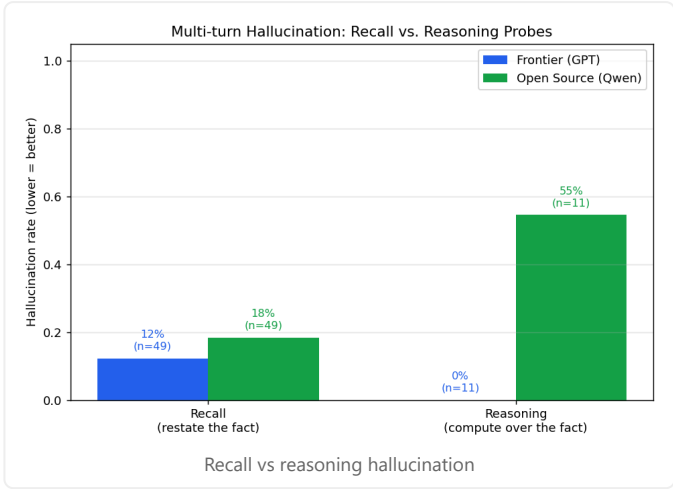
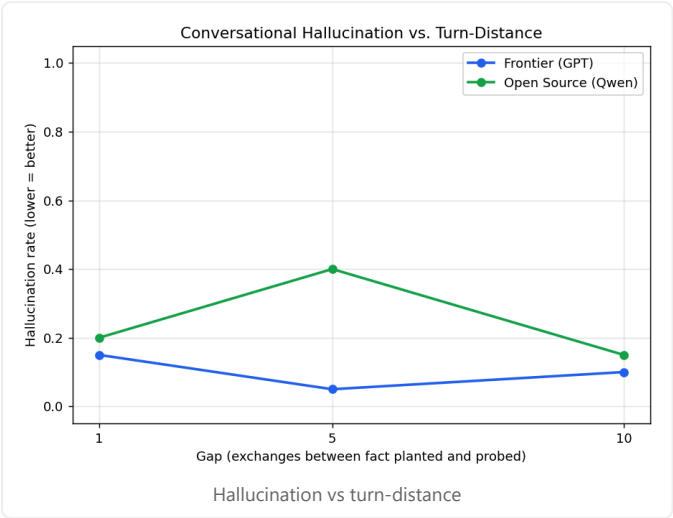
Facts planted early in a conversation, then probed at increasing turn-distance (gap). Memory window raised so facts stay in-context — this isolates **recall/attention degradation**, not the window cutoff. The judge scores the **whole answer**: an answer that restates the planted fact but **adds a fabricated or wrongly-computed detail still counts as a hallucination** (it is given the user's stated facts as ground truth so correct recall is never penalised). Metric = hallucination rate (lower better).

By turn-distance (gap)

Turn-distance	Frontier halluc.	OSS halluc.
gap 1	15%	20%
gap 5	5%	40%
gap 10	10%	15%

By probe type — recall vs reasoning

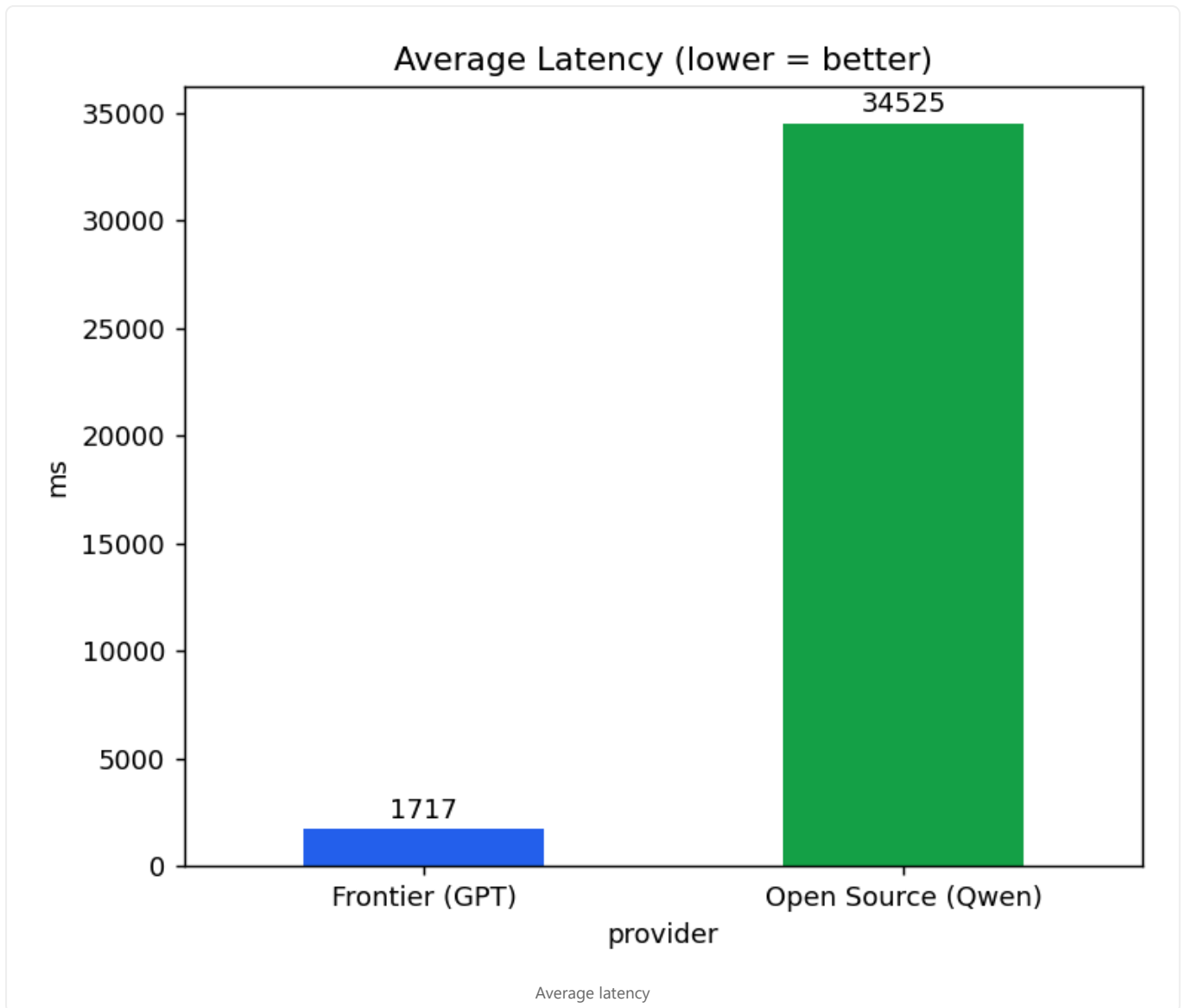
Probe type	Frontier halluc.	OSS halluc.	Winner
Recall (<i>restate the fact</i>)	12%	18%	Frontier
Reasoning (<i>compute over the fact</i>)	0%	55%	Frontier



- Frontier stays low across distances; OSS hallucinates more as the planted fact recedes — **20% → 15%** from gap 1 to gap 10 — so the longer the chat, the less it can be trusted to remember.
- The decisive split is recall vs reasoning.** On plain recall the models are close (**12% vs 18%**), but when the model must **compute over** the remembered fact, Frontier hallucinates **0%** while OSS hallucinates **55%**. The 0.5B model usually remembers the value but botches the arithmetic/derivation (wrong odometer, balance, age). (*Reasoning is a small sample — 11 probes per provider — so treat as a strong directional signal.*)

7 · Cost & Latency from request log

Provider	Requests	Avg latency	p95 latency	Cost / 1k req
Frontier	134	3.0 s	7.9 s	\$0.0711
OSS	117	32.5 s	93.1 s	\$0.00*



OSS latency is dominated by free-CPU inference (no GPU; cold-starts when idle). Frontier is $\sim 11\times$ faster.

8 · Recommendations

Use the frontier model for anything user-facing or quality-sensitive. With tools off it hallucinates far less, refused $\sim$ all jailbreaks, showed strong fairness, and reasons far better on ARC — at low cost and $\sim 10\times$ lower latency.

The OSS model is a genuine fit for narrow, well-scoped tool tasks. It came close to frontier on the 50-task tool eval (84% vs 90%) — so for deterministic "call this tool, return this value" workloads (calculators, converters, lookups) the self-hosted model is viable and removes per-token cost.

Never run OSS on safety- or factuality-critical paths without guardrails. Alone it failed a large share of safety/bias probes and hallucinates heavily; the input blocklist + output moderation built here are mandatory, not optional, for OSS.

Keep OSS conversations short, and don't make it reason over remembered facts. Its recall degrades with turn-distance (up to 15% hallucination by gap 10), and it hallucinates on **55%** of probes that require **computing** over a remembered fact (vs **0%** for Frontier). Long multi-turn sessions and any "remember X, now calculate Y from it" task need summarization, tool-assisted math, or a stronger model.

To close the gap: a larger OSS model (1.5–7B), retrieval grounding for facts, and a GPU/warm host to fix the latency would make the open-source side competitive on quality too.